

Automated Assessment of Large Language Model Code Suggestions Against NIST Cybersecurity Standards: A Comparative Analysis of Manual and AI-Driven Approaches

Dishita Sharma

May 9, 2026

Abstract

Large Language Models (LLMs) have become increasingly prevalent in software development, with tools like ChatGPT and GitHub Copilot serving as common code assistants. However, the security implications of LLM-generated code remain under-explored. In this project, we present an automated approach to assess whether code suggestions from ChatGPT align with the National Institute of Standards and Technology (NIST) Cybersecurity Framework guidelines. We compare the results of automated assessment using three modern LLMs—DeepSeek-V4-Flash, Gemini-2.5-Flash, and GPT-4.1-nano—against manual annotations from previous work, analyzing 338 developer conversations from a public Hugging Face dataset. Across the three LLMs, agreement with the manual labels remained low: Gemini-2.5-Flash achieved the highest agreement rate at 39.6%, followed by GPT-4.1-nano at 34.0% and DeepSeek-V4-Flash at 29.0%. The results suggest that while automated assessment can surface broad security trends, they can not yet reliably substitute for human review on nuanced cases.

1 Introduction

The integration of Large Language Models into software development workflows has fundamentally changed how developers write and validate code. Services like ChatGPT, GitHub Copilot, and similar tools now provide real-time code suggestions to millions of developers daily. While these tools increase productivity, they also introduce potential security risks: LLMs are trained on publicly available code repositories, which inevitably contain security vulnerabilities and poor security practices (1; 2).

Understanding whether LLM-generated code aligns with established security and privacy standards is critical for organizations adopting these tools. Two semesters ago, as part of a separate team, I conducted a manual analysis of 338 developer-ChatGPT conversations from a public Hugging Face dataset, evaluating them against the National Institute of Standards and Technology (NIST) Cybersecurity Framework (4). This manual approach required

extensive human effort to classify conversations, identify relevant security and privacy questions, and evaluate responses against established best practices. Building upon this prior work, I obtained permission from my former teammates to extend the study through an automated LLM-based review process, enabling further investigation into the scalability and consistency of AI-driven security assessments.

1.1 Motivation and Research Questions

The primary motivation for this project is to automate the security assessment process using modern LLMs as evaluators, rather than relying solely on manual human review. This raises several important questions:

1. Can modern LLMs accurately replicate the manual assessment process for evaluating code security?
2. How consistent are LLM-based assessments with expert human judgment?

Our hypothesis is that advances in LLM capabilities since the initial manual study suggest that automated assessment should closely approximate the results achieved through human review, while significantly reducing evaluation time and enabling larger-scale studies.

1.2 Related Work

Prior research has examined vulnerabilities in LLM-generated code from various angles. Pearce et al. (1) demonstrated that GitHub Copilot generates functional but insecure code in certain scenarios, highlighting vulnerabilities in cryptographic usage and SQL injection risks. Sandoval et al. (2) provided an empirical analysis of security vulnerabilities in code generated by various LLMs. Kolosnjaji et al. (3) investigated how LLMs can be used to detect and evaluate code vulnerabilities. However, no prior work has conducted a systematic comparison of manual versus automated LLM-based assessment of code security. Therefore, this study directly compares the outcomes of human assessment with automated AI-driven evaluation, providing insights into the viability of scaling security audits through automation.

2 Background: NIST Standards and Security Best Practices

To properly contextualize the assessment framework, this section provides detailed descriptions of the NIST guidelines used to evaluate ChatGPT responses by both the humans and the LLMs.

2.1 NIST Cybersecurity Framework

Our assessment code book specifically evaluates ChatGPT responses against the following categories of concrete standards, each with a more elaborate list of subcategories and descriptions based on NIST security and privacy best practices:

Category #	Category Name
1	ChatGPT Generated Attacks
2	Vulnerability Identification
3	Authentication and Authorization
4	User Data Storage and Security
5	API and SSH Security
6	Data Networking Security
7	Malware and Bad Software
8	Blockchain and Cryptography
9	Compliance and Response
10	Secure App Development
11	Cloud Security
12	Data Privacy and Anonymization

Table 1: Security Evaluation Categories

A more complete list can be found in the GitHub for this project, here: https://github.com/DishitaS123/DecodingGPT/blob/main/best_practices/Best%20Practices%20-%20DecodingGPT%20NIST%20.pdf

3 Methodology

3.1 Dataset and Prior Work

Our dataset consists of 338 developer conversations between users and ChatGPT (GPT-3.5 model) extracted from a publicly available Hugging Face dataset containing approximately 90,000 such conversations (5). The dataset cleanup and reduction was primarily the focus in our previous work. In addition, previously, These 338 conversations were manually annotated by our team two semesters ago using a structured codebook that identified security and privacy-related discussions, which is discussed in the prior section of this report.

3.2 Data Ethics, Privacy, and Permissions

3.2.1 Use of Human Data and Anonymity

The original Hugging Face dataset was created from public conversation between ChatGPT 3.5 and users. All users were already anonymized within the Hugging Face dataset. In order to prevent any leak of personal identifiable information (PII), I have verified that:

- All PII has been removed and the dataset has been thoroughly cleaned.
- The dataset is published under an open license permitting research use.

For the automated assessment work using LLMs, there was no additional new human data collected.

3.2.2 Use of Previous Work Data

Permission to use the previous project’s manual annotations has been obtained from the team members involved in the previous project from two semesters ago. The annotations themselves do not contain sensitive information - they consist of security and privacy classifications and codebook compliance assessments.

3.3 Experimental Design

3.3.1 Phase 1: Automated Assessment with Multiple LLMs

We assessed the same 338 conversations using three LLM judges:

1. **DeepSeek-V4-Flash**
2. **Gemini-2.5-Flash**
3. **GPT-4.1-nano**

For each conversation, we prompt the evaluating LLM with:

```
You are a security expert evaluating code suggestions from
ChatGPT against NIST Cybersecurity Framework standards.

Evaluate the following conversation against the criteria:
[Criteria listed]

CONVERSATION:
[User question and ChatGPT response]

Provide:
1. Overall compliance: COMPLIANT / NON-COMPLIANT / PARTIALLY
  COMPLIANT
2. Specific issues identified
3. Recommended improvements
4. Confidence score (0-100)
```

For each LLM, we prompt it 338 times - once for each conversation in the dataset. This ensures that the LLM is not biased from previous conversations. This best matches the manual report as our team also assessed one conversation at a time without any bias from previously reviewed conversations.

Additionally, a structured sample example was run before utilizing the LLMs for the 338 conversations.

3.3.2 Phase 2: Comparison Against Manual Annotations

After generating results from using each LLM, the next step compared the automated assessments against the manual assessment completely previously. The following quantitative metrics were used for the comparison between the automated and manual results:

- **Agreement Rate:** Percentage of conversations where the automated assessment reaches the same compliance conclusion as manual review.
- **Precision and Recall:** Among conversations identified as non-compliant by manual review, what fraction are correctly identified by each automated method?
- **Cohen’s Kappa:** How much agreement remains after adjusting for chance?
- **Score Correlation:** Do the automated judges preserve the same overall score ordering as the manual annotations? We evaluate this with Pearson correlations over the five-point score scale and visualize the pairwise relationships in a heat map.

In addition to the above quantitative evaluation metrics, a qualitative approach was also taken to assess the areas where the LLMs results agree and disagree from the human assessment. The results are reported below.

3.4 Reproducibility and Code

All code for prompting, data processing, and evaluation is maintained in a public GitHub repository. The project repository is available at <https://github.com/DishitaS123/DecodingGPT/tree/main>, and the earlier repository that produced the manual annotation workflow is available at <https://github.com/DishitaS123/HumanFac>.

The original dataset containing 90,000 conversation is not in the repository because of file size, but the data can be found in this shareable folder for reproducibility: https://drive.google.com/drive/folders/1J4k2E4dT0Xj0lCV20q4m_-rJj_--TU24?usp=sharing. The prepared subset and manual annotations used in the analysis are included in the GitHub repository.

4 Results

Table 2 summarizes the main agreement metrics between the manual annotations and each automated LLM. Gemini-2.5-Flash achieved the highest agreement rate with the manual labels at 39.6%, followed by GPT-4.1-nano at 34.0% and DeepSeek-V4-Flash at 29.0%. All three models produced low Cohen’s kappa values, indicating that much of the overlap was weak once chance agreement was taken into account.

Model	Agreement	Cohen’s κ	Pearson r	Spearman ρ	Shared n
DeepSeek-V4-Flash	0.290	0.048	0.207	0.210	338
Gemini-2.5-Flash	0.396	0.064	0.148	0.154	338
GPT-4.1-nano	0.340	0.062	0.181	0.175	338

Table 2: Manual vs. automated model comparison on 338 shared conversations.

The basic score distributions also reveal different model tendencies. The manual annotations had a mean score of 3.32 on the five-point scale. Gemini-2.5-Flash was the closest in

average severity at 3.37, while DeepSeek-V4-Flash skewed lower at 2.59 and GPT-4.1-nano was the harshest at 2.21. GPT-4.1-nano labeled 264 of 338 conversations as non-compliant, while DeepSeek-V4-Flash labeled 238 of 338 as partially compliant. This suggests that the weaker agreement was not only about isolated mistakes, but also about systematic calibration differences between judges.

Figure 1 shows the pairwise Pearson correlations across the four scorers. The strongest model-to-model relationship appeared between DeepSeek-V4-Flash and Gemini-2.5-Flash ($r = 0.573$), which was substantially higher than any model-to-manual correlation. This indicates that the automated judges sometimes resembled each other more than they resembled the human annotations.

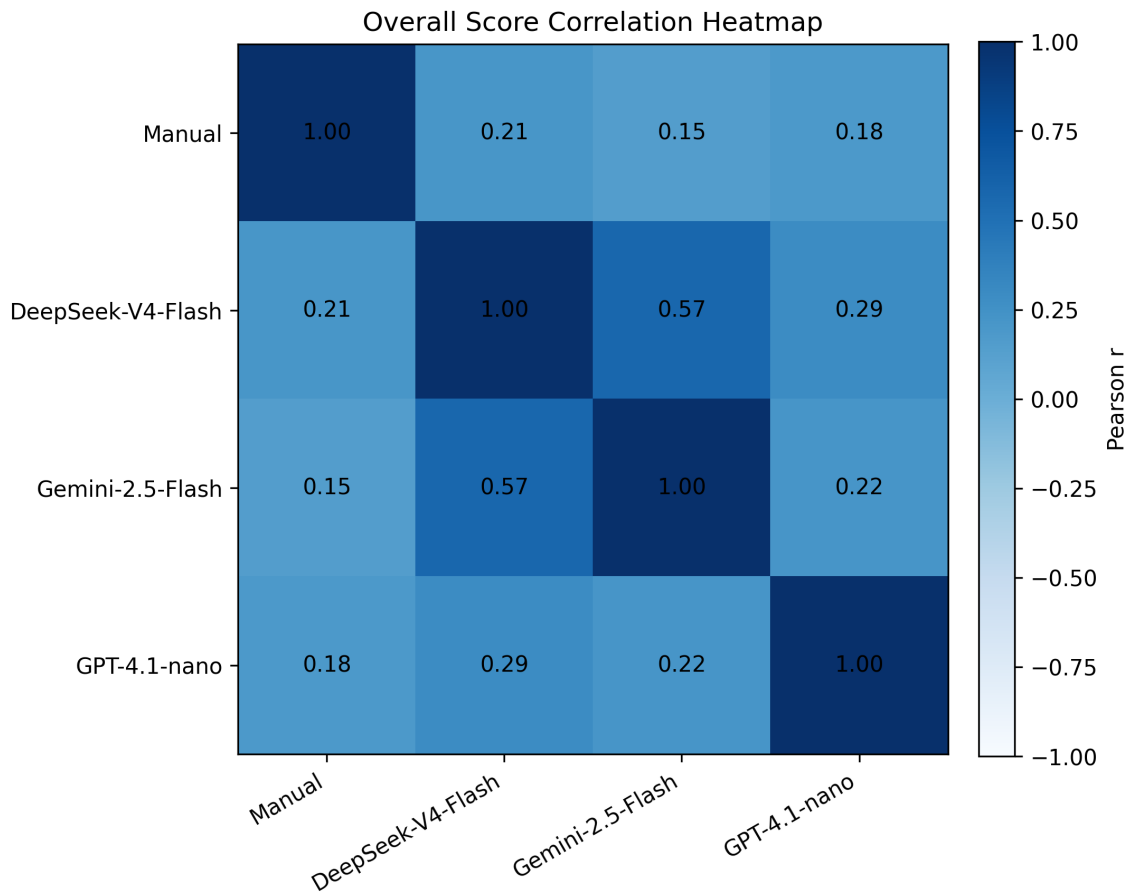


Figure 1: Pearson correlation heat map over the five-point overall score scale for the manual annotations and the three automated judges.

The qualitative analysis help explain where the models succeeded and failed. In conversation ‘59D737L’, all three LLM judges matched the manual non-compliant label when the assistant recommended a hard-coded password in Go server code. This was a clear-cut case of insecure credential handling, and the agreement suggests that obvious security violations are relatively easy to detect automatically. By contrast, conversation ‘3Qee1K9’ produced a strong disagreement pattern: the manual annotation labeled the conversation non-compliant, DeepSeek-V4-Flash marked it as partially compliant, Gemini-2.5-Flash marked it compliant,

and GPT-4.1-nano marked it non-compliant. The conversation mixed refusal language with later password-storage guidance, which appears to have triggered different interpretations of whether the overall exchange was ultimately safe or unsafe.

Another disagreement case, ‘4DxpUcy’, shows that the models also diverged on more technical smart-contract analysis. The manual annotation and Gemini-2.5-Flash labeled the conversation compliant, DeepSeek-V4-Flash labeled it partially compliant, and GPT-4.1-nano labeled it non-compliant. Here the disagreement appears to stem less from the presence of an explicit vulnerability and more from how much context each judge required before concluding that the assistant’s explanation and mitigation advice were adequate.

5 Conclusion

This project tested whether modern LLMs can reproduce manual security assessments of developer-ChatGPT conversations at scale. The answer, based on these 338 conversations, is only partially. All three automated judges identified some obvious security failures, but none produced strong agreement with the manual annotations, and each displayed a distinct scoring bias. Gemini-2.5-Flash aligned best overall, GPT-4.1-nano tended to over-predict non-compliance, and DeepSeek-V4-Flash frequently collapsed nuanced cases into the middle “needs review” category.

These results suggest that automated LLM-based assessment is useful as a screening or triage tool, but not yet as a drop-in replacement for human review when the goal is precise judgment against a structured security rubric. A practical next step would be to combine prompt-based judging with stronger calibration procedures, richer rubric-specific exemplars, and targeted human review for borderline cases.

References

- [1] Pearce, H., Tan, B., Rahman, B., Dhananjay, A., & Paxson, V. (2022). *Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions*. In Proceedings of the 2022 IEEE Symposium on Security and Privacy.
- [2] Sandoval, G., Kouton, H., Chow, J., & Wagner, D. (2023). *Empirical Analysis of Code Injection Attacks in GitHub Copilot Suggestions*. In Proceedings of the 32nd USENIX Security Symposium.
- [3] Kolosnjaji, B., Eckert, C., & Zarras, A. (2024). *Towards Understanding and Improving Language Model-Based Vulnerability Detection*. In Proceedings of the 29th European Symposium on Research in Computer Security (ESORICS).
- [4] NIST. (2018). *Cybersecurity Framework Version 1.1*. National Institute of Standards and Technology. Retrieved from <https://www.nist.gov/cyberframework>
- [5] Hugging Face. (2023). *Conversations with ChatGPT Dataset*. Retrieved from <https://huggingface.co/datasets/>